ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО	УТВЕРЖДЕНО
Приглашенный преподаватель	Академический руководитель образовательной программы "Программная инженерия", старший преподаватель департамента программной инженерии
_____ С. Б. Гурин	_____ Н. А. Павлочев
«___» _____ 2025 г.	«___» _____ 2025 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

SENTINEL — ИНТЕЛЛЕКТУАЛЬНЫЙ ТАЙМ-МЕНЕДЖЕР

Техническое задание

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.06.05-01 ТЗ 01-1-ЛУ

Исполнители:

Студент группы БПИ236
_____ / Т. А. Петянов /
«___» _____ 2025 г.

Студент группы БПИ236
_____ / Т. С. Максимов /
«___» _____ 2025 г.

2025

УТВЕРЖДЕН

RU.17701729.06.05-01 ТЗ 01-1-ЛУ

SENTINEL — ИНТЕЛЛЕКТУАЛЬНЫЙ ТАЙМ-МЕНЕДЖЕР

Техническое задание

RU.17701729.06.05-01 ТЗ 01-1

Листов 33

2025

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

АННОТАЦИЯ

Техническое задание – это основной документ, оговаривающий набор требований и порядок создания программного продукта, в соответствии с которым производится разработка программы ее тестирование и приемка.

Настоящее Техническое задание на разработку «Sentinel — Интеллектуальный тайм-менеджер» содержит следующие разделы: «Введение», «Основания для разработки», «Назначение разработки», «Требования к программе», «Требования к программной документации», «Технико-экономические показатели», «Стадии и этапы разработки», «Порядок контроля и приемки», приложения [7].

В разделе «Введение» указано наименование и краткая характеристика области применения программы.

В разделе «Основания для разработки» указан документ, на основании которого ведется разработка, и наименование темы разработки.

В разделе «Назначение разработки» указано функциональное и эксплуатационное назначение создаваемого программного продукта.

Раздел «Требования к программе» содержит указание на основные требования к функциональным характеристикам программы, к её надежности и к условиям эксплуатации, к составу и параметрам технических средств, к информационной и программной совместимости, к маркировке и упаковке, к транспортировке и хранению, а также специальные требования.

Раздел «Требования к программным документам» содержит указание на предварительный состав программной документации и специальные требования к ней.

Раздел «Технико-экономические показатели» содержит информацию об ориентировочной экономической эффективности разработки, экономические преимущества разработки программы.

Раздел «Стадии и этапы разработки» содержит информацию о стадиях разработки, этапах и содержании работ.

В разделе «Порядок контроля и приемки» указаны общие требования к приемке работы.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 [1]: Виды программ и программных документов.
2. ГОСТ 19.102-77 [2]: Стадии разработки.
3. ГОСТ 19.103-77 [3]: Обозначения программ и программных документов.
4. ГОСТ 19.104-78 [4]: Основные надписи.
5. ГОСТ 19.105-78 [5]: Общие требования к программным документам.

6. ГОСТ 19.106-78 [6]: Требования к программным документам, выполненным печатным способом.
7. ГОСТ 19.201-78 [7]: Техническое задание. Требования к содержанию и оформлению.

Изменения к данному Техническому заданию оформляются согласно ГОСТ 19.603-78 [12], ГОСТ 19.604-78 [13].

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ . . . 7
1.1. Наименование программы . . . 7
1.2. Краткая характеристика области применения программы . . . 7
2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ . . . 8
2.1. Документ(ы), на основании которого(ых) ведется разработка . . . 8
2.2. Наименование темы разработки . . . 8
3. НАЗНАЧЕНИЕ РАЗРАБОТКИ . . . 9
3.1. Функциональное назначение . . . 9
3.2. Эксплуатационное назначение . . . 9
4. ТРЕБОВАНИЯ К ПРОГРАММЕ . . . 10
4.1. Требования к функциональным характеристикам . . . 10
4.1.1. Требования к составу выполняемых функций . . . 10
4.1.2. Требования к организации входных данных . . . 12
4.1.3. Требования к организации выходных данных . . . 13
4.1.4. Требования к временным характеристикам . . . 14
4.1.5. Требования к интерфейсу . . . 14
4.2. Требования к надежности . . . 14
4.2.1. Требования к обеспечению надежного функционирования . . . 14
4.2.2. Требования к восстановлению при отказах . . . 15
4.2.3. Обработка ошибок и отказоустойчивость . . . 15
4.2.4. Требования к надежности (серверная часть) . . . 15
4.3. Условия эксплуатации . . . 15
4.3.1. Климатические условия эксплуатации . . . 15
4.3.2. Требования к видам обслуживания . . . 15
4.3.3. Требования к численности и квалификации персонала . . . 16
4.4. Требования к составу и параметрам технических средств . . . 16
4.4.1. Программные средства . . . 16
4.4.2. Программные средства (серверная часть) . . . 16
4.4.3. Технические средства (серверная часть) . . . 16
4.4.4. Технические средства . . . 16
4.5. Требования к информационной и программной совместимости . . . 17

4.5.1. Требования к информационным структурам и методам решения 17
4.5.2. Интерфейсы API (серверная часть) 18
4.5.3. Требования к программным средствам, используемым программой 18
4.5.4. Требования к исходным кодам и языкам программирования 18
4.5.5. Требования к защите информации и программы 18
4.6. Требования к маркировке и упаковке 19
4.7. Требования к транспортированию и хранению 20
4.8. Специальные требования 20
5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ 21
5.1. Состав программной документации 21
5.2. Специальные требования к программной документации 21
6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ 22
6.1. Предполагаемая потребность 22
6.2. Целевая аудитория 22
6.3. Преимущества перед аналогами 22
6.3.1. Анализ аналогов 22
6.3.1.1. Motion 22
6.3.1.2. Reclaim.ai 22
6.3.1.3. Sunsama 23
6.3.1.4. Things 3 23
6.3.1.5. Todoist 23
6.3.1.6. Notion Calendar 23
6.3.1.7. Sentinel 24
6.3.2. Дифференциаторы «Sentinel» 24
6.3.3. Сравнительные критерии (матрица) 24
7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ 26
7.1. Стадии разработки, этапы и содержание работ 26
7.2. Сроки разработки и исполнители 28
8. ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ 29
8.1. Виды испытаний 29
8.2. Общие требования к приемке работы 29
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ 30

ПРИЛОЖЕНИЕ А. ТЕРМИНОЛОГИЯ 31
ПРИЛОЖЕНИЕ Б. ССЫЛКИ НА АНАЛОГИ 32

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы: «Sentinel — Интеллектуальный тайм-менеджер».
Наименование программы на английском языке: «Sentinel — AI Scheduler».
Краткое наименование программы для пользователя: «Sentinel».

1.2. Краткая характеристика области применения программы

«Sentinel — Интеллектуальный тайм-менеджер» — iOS-приложение для персонального тайм-менеджмента с чат-интерфейсом. Система преобразует разрозненные входы (короткие текстовые запросы, фрагменты писем, фотографии расписаний и списков дел, документы) в структурированные календарные события. Интеграция с внешними OCR/LLM-сервисами позволяет извлекать задачи, сроки и приоритеты без необходимости ручного разметки.

На практике приложение снимает барьер между «сырой» информацией и планом на день: пользователь формулирует намерение на естественном языке и, при необходимости, прикрепляет изображение или документ; система предлагает набор событий, учитывая индивидуальный контекст (модель «Resource Battery») и подсказывает оптимальный порядок (Rebalance).

Области применения включают:

1. Ежедневное планирование учёбы и сессии (лекции, семинары, дедлайны работ);
2. Проектную работу и распределение времени между исследованием, разработкой и встречами;
3. Личные дела и бытовые задачи, требующие напоминаний и времени на переключения;
4. Подготовку к мероприятиям (экзамены, презентации, собеседования) с разбивкой на этапы;
5. Координацию встреч и блоков «глубокой работы»;
6. Ведение совмещённого расписания на iPhone/iPad с системными уведомлениями.

2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ

2.1. Документ(ы), на основании которого(ых) ведется разработка

Разработка ведётся на основании учебного плана подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утвержденной академическим руководителем программы темы курсового проекта.

2.2. Наименование темы разработки

Наименование темы разработки: «Sentinel — Интеллектуальный тайм-менеджер (Клиентская часть)».

Условное обозначение темы разработки: «Sentinel».

3. НАЗНАЧЕНИЕ РАЗРАБОТКИ

3.1. Функциональное назначение

Программа обеспечивает генерацию календарных событий на основе диалога пользователя с ассистентом в экране «Чат». Пользователь формулирует запросы на естественном языке, прикрепляет при необходимости фотографии (PNG/JPG/HEIC) и документы (PDF, DOCX, TXT, CSV, MD, PPTX, XLSX, JSON), получает от ассистента предложения и отмечает нужные события для добавления. Добавление событий выполняется в календарь устройства.

Система учитывает индивидуальные ограничения и контекст пользователя через модель «Resource Battery», поддерживает автоматическую балансировку порядка событий (Rebalance) и ведёт учёт достижений. Приложение использует EventKit для синхронизации с календарями, хранит добавленные события и базовые настройки локально и взаимодействует с серверной частью по HTTPS/JSON.

3.2. Эксплуатационное назначение

Программа предназначена для повышения персональной продуктивности и качества планирования времени на устройствах iOS. Целевыми пользователями являются студенты и специалисты, которым требуется быстро преобразовывать произвольные запросы и вложения в набор календарных событий.

Для интерпретации запросов и вложений необходимо активное подключение к сети Интернет; генерация событий ассистентом недоступна в офлайн-режиме, при отсутствии подключения доступен просмотр ранее добавленных событий. Требуются базовые навыки работы с iOS.

Серверная часть — требования к эксплуатации:

1) Развёртывание в контейнерах (Docker) и доступ по защищённым REST-эндпоинтам.
2) Инфраструктура с Python 3.11+, FastAPI и PostgreSQL 14+.
3) Доступ к Интернету для внешних LLM-вызовов (OpenAI SDK).
4) Корректный учёт часовых поясов пользователя (IANA).
5) Администрирование и сопровождение разработчиком/DevOps.

4. ТРЕБОВАНИЯ К ПРОГРАММЕ

4.1. Требования к функциональным характеристикам

4.1.1. Требования к составу выполняемых функций

iOS-приложение должно реализовывать следующие функции:

1. Экран «Чат»
    1.1. Диалог с ассистентом
        1.1.1. Отправка текстовых сообщений.
        1.1.2. Прикрепление фотографий (PNG/JPG/HEIC) из галереи и камеры.
        1.1.3. Прикрепление документов (PDF/DOCX/TXT/CSV/MD/PPTX/XLSX/JSON) через системный диалог.
        1.1.4. Отображение ошибок отправки/обработки с кнопкой «Повторить».
    1.2. Генерация событий календаря
        1.2.1. Получение предложений событий от ассистента.
        1.2.2. Отметка предложенных событий для добавления (множественный выбор).
        1.2.3. Добавление выбранных событий в календарь.
2. Экран «Сегодня»
    2.1. Представление дня
        2.1.1. Отображение списка событий на текущий день.
        2.1.2. Индикатор «Resource Battery».
        2.1.3. Кнопка «Создать событие» для перехода в «Чат».
    2.2. Подсказки и балансировка
        2.2.1. Быстрое добавление событий из предложений чата.
        2.2.2. Запуск автоматической балансировки расписания (Rebalance).
        2.2.3. Отображение прогресса по системе достижений.
3. Экран «Календарь»
    3.1. Просмотр расписания
        3.1.1. Представление недели и месяца.
        3.1.2. Синхронизация с Apple Calendar через EventKit.
    3.2. Создание события
        3.2.1. Создание события в календаре.
4. Экран «Настройки»
    4.1. Учётная запись и уведомления

4.1.1. Просмотр статуса авторизации и переход к экрану «Авторизация».

4.1.2. Выход из учётной записи.

4.1.3. Включение/выключение локальных уведомлений.

4.1.4. Удаление учётной записи с модальным подтверждением удаления.

4.2. Достижения

4.2.1. Переход на экран «Достижения».

4.3. Глобальные правила (дефолтный промпт)

4.3.1. Ввод и сохранение шаблона, автоматически добавляемого ко всем запросам ассистенту.

5. Экран «Достижения»

5.1. Просмотр и совместное использование

5.1.1. Просмотр выданных достижений.

5.1.2. Просмотр доступных достижений.

5.1.3. Поделиться достижением через системное окно Share Sheet.

6. Экран «Авторизация»

6.1. Вход через Sign in with Apple (ASAuthorizationAppleID).

6.2. Отображение политики конфиденциальности и согласия на обработку данных.

6.3. Отображение ошибок авторизации и кнопка «Повторить».

7. Серверная часть (бэкенд)

7.1. Обработка диалоговых запросов

7.1.1. Оркестрация LLM (OpenAI SDK) с использованием function calling/tool-use.

7.1.2. Определение намерений пользователя и формирование структурированных запросов к данным.

7.2. Управление календарём

7.2.1. Поиск по временным диапазонам («сегодня», «завтра», «на неделе», произвольный интервал) и по ключевым словам.

7.2.2. Создание, обновление и удаление событий и напоминаний (напоминание трактуется как событие без времени окончания).

7.3. Управление чатами

7.3.1. Ведение чат-сессий и сообщений, хранение текстовой истории взаимодействий.

7.3.2. Протоколирование вызовов инструментов (аргументы/результаты) при необходимости.

7.4. Аутентификация и авторизация

7.4.1. Регистрация и вход, выдача и проверка JWT; доступ строго в пределах владельца данных.

7.5. Совместимость и документация

7.5.1. REST API с описанием в OpenAPI 3.x.

7.5.2. Использование ISO-8601 для дат/времени и учёт часового пояса пользователя (IANA).

7.6. Достижения

7.6.1. Выдача пользовательских достижений на основании заданных правил.

4.1.2. Требования к организации входных данных

Входные данные представляют собой:

1. Сообщения в чате

1.1. Текст до 2000 символов в кодировке UTF-8.

1.2. Фотографии форматов PNG/JPG/JPEG/HEIC; до 10 МБ за файл; до 5 фото на сообщение.

1.3. Документы форматов PDF/DOCX/TXT/CSV/MD/PPTX/XLSX/JSON; до 10 МБ за файл; до 3 документов на сообщение.

2. Правила приёма и валидации

2.1. Проверка длины сообщения и допустимых символов.

2.2. Проверка типа, размера и количества фотографий.

2.3. Проверка типа, размера и количества документов.

2.4. Согласие на серверную обработку текста, фотографий и документов.

3. Обработка ошибок

3.1. Отображение ошибки сети с кнопкой «Повторить».

3.2. Повторный запрос к ассистенту по инициативе пользователя.

4. Серверная часть (входные данные)

4.1. Формат обмена: JSON по HTTPS.

4.2. Аутентификация: заголовок Authorization: Bearer для защищённых эндпоинтов.

4.3. Даты/время: ISO-8601 (UTC) с учётом пользовательской тайм-зоны (IANA) на уровне сервиса.

4.4. Диалоговые сообщения: текстовые запросы, структурированные данные инструмента (tool-args), идентификаторы чат-сессий.

4.5. События/напоминания: title, start_at, end_at?, all_day, type (event/reminder), location?, tags?.

4.6. Ограничения: валидация схемы входных данных и проверка прав доступа к ресурсам пользователя.

4.1.3. Требования к организации выходных данных

1. Внутренние сущности

1.1. Пользователь — владелец учётной записи и персональных настроек, в том числе предпочтений расписания и часового пояса.

1.2. Чат — логичная переписка пользователя с ассистентом, в которой формулируются запросы и предлагаются события.

1.3. Сообщение чата — отдельное высказывание пользователя или ассистента; может сопровождаться структурированными данными для создания событий.

1.4. Событие — запись календаря с названием, временем, длительностью и дополнительными пометками; создаётся из предложений ассистента или вручную.

1.5. Достижение пользователя — отметка о достижении устойчивых полезных паттернов планирования с уровнем и описанием.

2. Внешние артефакты

2.1. События календаря iOS: заголовок, заметки, напоминания (без вложений изображений/ документов).

2.2. Ответы сервера: текст и структурированные данные (JSON).

2.3. Локальные уведомления (UNUserNotificationCenter) для напоминаний, управляемых приложением.

3. Целостность данных

3.1. Исключение дублирования событий при экспорте.

3.2. Визуальная индикация конфликтов расписания.

4. Серверные ответы (выходные данные)

4.1. Формат: JSON; коды ответов HTTP соответствуют семантике (2xx/4xx/5xx).

4.2. Поиск событий: массив items[] с полями id, type (event/reminder), title, start_at, end_at?, location?, source; при необходимости — summary.

4.3. Ответы по чатам: хронологический список сообщений с ролями (user/assistant/tool/system) и техническими полями (опц.).

4.4. Идентификаторы: UUID (строка).

4.5. Ошибки: единый формат {code, message, details}.

4.1.4. Требования к временным характеристикам

1. Время отклика интерфейса — не более 150 мс.

2. Переключение экранов — не более 300 мс.

3. Интерпретация запроса на сервере — 3–6 секунд (таймаут 15 секунд).

4. Создание события в календаре — не более 500 мс.

5. Фоновые операции не используются; незавершённые операции предлагается повторить вручную после возвращения в приложение.

6. Сервер: простые операции чтения ≤ 500 мс; поиск по БД (диапазон + текст) — медиана ≤ 200–300 мс; синхронный диалог с LLM — ответ ≤ 8 с; вставка/обновление записи — ≤ 300 мс.

4.1.5. Требования к интерфейсу

1. Соответствие Apple Human Interface Guidelines; поддержка светлой/тёмной темы и Dynamic Type.

2. Экраны и элементы:

2.1. «Чат» — поле ввода, кнопка отправки, лента сообщений, выбор фотографий (камера/галерея), выбор документов через системный диалог.

2.2. «Календарь» — список/сетка событий дня, недели и месяца.

3. Минимальный размер интерактивных элементов — 44×44 pt.

4. Доступность: поддержка VoiceOver; корректные accessibility-метки; контраст не ниже WCAG 2.1 AA.

5. Индикация состояний: отображение прогресса сетевых операций, текст ошибки и кнопка «Повторить».

6. Локализация: ru-RU; формат даты/времени и чисел следует региональным настройкам iOS.

4.2. Требования к надежности

4.2.1. Требования к обеспечению надежного функционирования

1. Устойчивость работы приложения

1.1. Crash-free не ниже 99,5% на поддерживаемых устройствах.

1.2. Контроль памяти до 250 МБ в типовых сценариях и защита от аварий при обработке больших изображений.

1.3. Холодный запуск приложения не более 2 секунд.

1.4. Валидация входных данных перед обработкой.

1.5. Идемпотентность операций с календарём для предотвращения дублирования.

4.2.2. Требования к восстановлению при отказах

1. Время восстановления

1.1. Автоматическое восстановление последнего стабильного состояния при перезапуске.

1.2. Автосохранение данных перед сетевыми операциями.

4.2.3. Обработка ошибок и отказоустойчивость

1. Сетевые сбои и таймауты

1.1. Ручной повтор запроса пользователем.

1.2. Деградация при недоступности AI.

4.2.4. Требования к надежности (серверная часть)

1. Идемпотентность операций создания событий (поддержка X-Idempotency-Key на ограниченное окно времени).

2. Тайм-ауты и повторы с джиттером для внешних LLM-вызовов; graceful degradation.

3. Контроль целостности данных: транзакции БД для критичных операций; валидация на уровне схем.

4. Логи и базовый аудит ключевых действий (создание/изменение/удаление).

5. Целевая доступность сервиса — не ниже 99.5% (ориентир учебного проекта).

4.3. Условия эксплуатации

4.3.1. Климатические условия эксплуатации

Эксплуатация соответствует климатическим требованиям устройств Apple: температура 0…35 °C; относительная влажность 5–95% без конденсации.

Программа обеспечивает работоспособность как в стационарных условиях, так и в мобильных условиях при наличии устойчивого интернет-соединения.

4.3.2. Требования к видам обслуживания

1. Пользовательское обслуживание:

1.1. Своевременное обновление приложения через App Store или средства тестирования.

1.2. Управление разрешениями доступа к системным ресурсам.

1.3. Наличие свободного места и достаточного заряда аккумулятора.

2. Техническое обслуживание:

2.1. Регулярное резервное копирование исходного кода и артефактов сборки.

2.2. Тестирование на поддерживаемых устройствах и симуляторах.

4.3.3. Требования к численности и квалификации персонала

1. Пользователи:

1.1. Численность: не ограничена.

1.2. Квалификация: базовые навыки работы с операционной системой iOS, понимание принципов предоставления разрешений приложениям.

2. Персонал разработки и сопровождения:

2.1. Квалификация: знание языка Swift, фреймворков SwiftUI, EventKit, принципов работы с REST API и OAuth.

4.4. Требования к составу и параметрам технических средств

4.4.1. Программные средства

Для функционирования программы необходимы:

1. Операционная система: iOS версии 26.0 и выше.
2. Среда выполнения: устройства Apple iPhone или iPad.

Для разработки программы используются:

1. Язык программирования: Swift 6.
2. Среда разработки: Xcode 26.x.
3. Фреймворки: SwiftUI, Combine, EventKit, PhotosUI, UserNotifications, Keychain, Foundation.
4. Система сборки и управления зависимостями: Swift Package Manager.

4.4.2. Программные средства (серверная часть)

1) Язык/платформа: Python 3.11+.

2) Фреймворк: FastAPI (ASGI: Uvicorn/Gunicorn).

3) Доступ к БД: SQLAlchemy 2.0 (async) + драйвер asyncpg; миграции — Alembic.

4) Валидация/схемы: Pydantic v2.

5) База данных: PostgreSQL 14+.

6) LLM-клиент: OpenAI SDK (function calling/tool-use).

7) Контейнеризация: Docker (для сборки/развёртывания).

4.4.3. Технические средства (серверная часть)

1) Минимальные требования (dev/test): CPU 2 vCPU, RAM 4 ГБ, SSD 20 ГБ, доступ в Интернет.

2) Продуктивное окружение масштабируется по нагрузке (например, 4 vCPU, 8 ГБ RAM).

4.4.4. Технические средства

1. Мобильные устройства:

1.1. iPhone: модели, официально поддерживаемые Apple для iOS 26.

1.2. iPad: модели, официально поддерживаемые Apple для iPadOS 26.

2. Ресурсы устройства:

2.1. Оперативная память не менее 4 ГБ.

2.2. Свободное дисковое пространство не менее 500 МБ.

2.3. Наличие камеры.

3. Сеть:

3.1. Подключение к сети Интернет с пропускной способностью, достаточной для обмена данными.

4.5. Требования к информационной и программной совместимости

4.5.1. Требования к информационным структурам и методам решения

1. Модель данных:

1.1. Локальная база данных для хранения событий, настроек и истории чата.

1.2. Основные сущности: Event, ChatMessage, ChatThread, Settings.

1.3. Хранилище: SwiftData; версионирование схемы и миграции включены.

2. Протоколы обмена:

2.1. Взаимодействие с сервером по HTTPS.

2.2. Формат обмена данными JSON в UTF-8.

2.3. Дата и время в ISO-8601 с указанием часового пояса.

3. Совместимость с календарём:

3.1. Соответствие полям Apple Calendar: заголовок, заметки, напоминания, идентификатор календаря.

3.2. Использование уникальных идентификаторов для синхронизации.

3.3. Идемпотентность: внешний идентификатор события (UUID) хранится в EKEvent.url и/или заметках; при добавлении/обновлении выполняется дедупликация по этому идентификатору.

4. Протоколы обмена (сервер):

4.1. JSON по HTTPS; кодировка UTF-8.

4.2. Версионирование REST: префикс /api/v1; контракты — OpenAPI 3.x.

4.3. Аутентификация: JWT в заголовке Authorization: Bearer .

5. Схема данных и БД (сервер):

5.1. СУБД: PostgreSQL 14+; идентификаторы — UUID; время — TIMESTAMPTZ (UTC).

5.2. Таблицы: users, chats, chat_messages, events, user_achievements.

5.3. Базовые индексы: users(email) UNIQUE; events(user_id, start_at); chat_messages(chat_id, created_at).

4.5.2. Интерфейсы API (серверная часть)

1) Аутентификация:

1.1) POST /api/v1/auth/register — регистрация пользователя.

1.2) POST /api/v1/auth/login — вход, выдача JWT.

2) Диалоги/чаты:

2.1) GET /api/v1/chats — список чатов.

2.2) POST /api/v1/chats — создать чат-сессию.

2.3) GET /api/v1/chats/{id}/messages — история сообщений.

2.4) POST /api/v1/chats/{id}/messages — отправить сообщение.

2.5) POST /api/v1/llm/chat — оркестратор диалога (system prompt, tools, tool-use).

3) События:

3.1) GET /api/v1/events?range=…&q=… — поиск по диапазону и строке.

3.2) POST /api/v1/events — создать событие/напоминание.

3.3) PATCH /api/v1/events/{id} — обновить событие/напоминание.

3.4) DELETE /api/v1/events/{id} — удалить событие/напоминание.

4) Достижения:

4.1) GET /api/v1/achievements — достижения текущего пользователя.

4.5.3. Требования к программным средствам, используемым программой

Программа использует стандартные библиотеки и фреймворки, предоставляемые Apple SDK. Сторонние зависимости подключаются через Swift Package Manager. Использование динамической загрузки кода запрещено.

4.5.4. Требования к исходным кодам и языкам программирования

1. Язык программирования: Swift.
2. Архитектура: TCA (The Composable Architecture) с однонаправленным потоком данных.
3. Соблюдение Swift API Design Guidelines.
4. Наличие модульной структуры проекта.

4.5.5. Требования к защите информации и программы

1. Сетевая безопасность:

1.1. Все внешние запросы выполняются по HTTPS (TLS ≥ 1.2).

1.2. Используется App Transport Security (ATS); небезопасные соединения запрещены

(NSAppTransportSecurity.NSAllowsArbitraryLoads = false).

1.3. Запрет HTTP; проверка цепочки сертификатов по системным корневым центрам сертификации.

1.4. Сертификат-пиннинг не используется в первой версии; допускается включение в будущих релизах.

2. Хранение данных:

2.1. Токены доступа и ключи хранятся в Keychain.

2.2. Минимизируется объём сохраняемых персональных данных.

2.3. Чувствительная информация исключается из логов.

3. Приватность:

3.1. Разрешения запрашиваются контекстно, с пояснением цели.

3.2. Info.plist включает ключи: NSCameraUsageDescription, NSPhotoLibraryUsageDescription, NSCalendarsUsageDescription, при необходимости — NSPhotoLibraryAddUsageDescription.

3.3. Уведомления запрашиваются через UNUserNotificationCenter; используются только локальные уведомления.

4. Удаление учётной записи:

4.1. В «Настройках» доступен сценарий удаления учётной записи и локальных данных; при наличии сети отправляется запрос на деактивацию учётной записи на сервере в соответствии с требованиями App Store.

5. Безопасность сервера:

5.1. Аутентификация по JWT; пароли хранятся только в виде безопасных хешей.

5.2. Секреты и ключи — в переменных окружения/secret store.

5.3. Защита от базовых атак: rate-limit, валидация входных данных.

4.6. Требования к маркировке и упаковке

1. Маркировка:

1.1. Идентификатор пакета com.example.sentinel.

1.2. Версионирование по Semantic Versioning.

1.3. Отображаемое имя «Sentinel».

1.4. Комплект иконок для всех поддерживаемых устройств и тем.

1.5. Локализация интерфейса и метаданных.

2. Упаковка:

2.1. Исходный код в репозитории системы контроля версий.

2.2. Инструкция по сборке и запуску.

2.3. Архив сборки для демонстрации.

2.4. Проектная документация в формате PDF.

4.7. Требования к транспортированию и хранению

1. Хранение исходного кода:

1.1. Приватный репозиторий Git.

1.2. Регулярные резервные копии репозитория.

1.3. Срок хранения не менее 5 лет.

2. Хранение пользовательских данных:

2.1. Локальное хранение базы данных на устройстве.

2.2. Удаление всех локальных данных средствами приложения.

4.8. Специальные требования

1. Конфиденциальность:

1.1. Обработка текстовых сообщений, фотографий и документов выполняется на сервере; пользователь должен быть явно проинформирован об этом и дать согласие.

2. Доступность:

2.1. Поддержка технологий вспомогательного доступа iOS.

2.2. Поддержка динамического размера шрифта.

2.3. Достаточный контраст элементов интерфейса.

3. Энергоэффективность:

3.1. Фоновая обработка приложением не используется; сетевые операции выполняются только в активном режиме.

3.2. Эффективные алгоритмы обработки данных.

5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

5.1. Состав программной документации

1. «Sentinel — Интеллектуальный тайм-менеджер». Техническое задание (ГОСТ 19.201-78 [7]).
2. «Sentinel — Интеллектуальный тайм-менеджер». Пояснительная записка (ГОСТ 19.404-79 [10]).
3. «Sentinel — Интеллектуальный тайм-менеджер». Программа и методика испытаний (ГОСТ 19.301-79 [8]).
4. «Sentinel — Интеллектуальный тайм-менеджер». Текст программы (ГОСТ 19.401-78 [9]).
5. «Sentinel — Интеллектуальный тайм-менеджер». Руководство оператора (ГОСТ 19.505-79 [11]).

5.2. Специальные требования к программной документации

Документы к программе должны быть выполнены в соответствии с ГОСТ 19.106-78 и ГОСТами к каждому виду документа (см. п. 5.1.); Пояснительная записка должна быть загружена в систему Антиплагиат через LMS «НИУ ВШЭ».

Техническое задание и пояснительная записка, титульные листы других документов должны быть подписаны руководителем разработки и исполнителем.

Документация и программа сдается в электронном виде в формате .pdf или .docx. в архиве формата .zip или .rar;

За две недели до защиты комиссии все материалы курсового проекта: программная документация, программный проект, исполняемый файл, отзыв руководителя, отчет системы Антиплагиат, должны быть загружены одним или несколькими архивами в проект дисциплины «Курсовой проект» в личном кабинете в информационной образовательной среде SmartLMS НИУ ВШЭ.

6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

6.1. Предполагаемая потребность

Программа удовлетворяет потребность пользователей в быстрой конвертации текстовых запросов в структурированные события календаря. Использование технологий искусственного интеллекта позволяет автоматизировать извлечение атрибутов событий (дедлайны, приоритеты), сокращая время на планирование.

6.2. Целевая аудитория

Целевой аудиторией программы являются студенты и специалисты, активно использующие мобильные устройства Apple (iPhone, iPad) для планирования своего времени и нуждающиеся в инструментах повышения личной продуктивности.

6.3. Преимущества перед аналогами

Разрабатываемое приложение «Sentinel» решает задачу генерации календарных событий на основе текстового диалога с ассистентом и дополнительно учитывает индивидуальные ограничения пользователя через модель «Resource Battery», автоматическую балансировку «Rebalance» и систему достижений. Ниже приведён взвешенный анализ рынка.

6.3.1. Анализ аналогов

6.3.1.1. Motion

Плюсы:

1. Автопланирование задач и пересборка расписания по приоритетам.
2. Интеграция с основными календарями и таск-системами.
3. Инструменты для командной работы и слотов для встреч.

Минусы:

1. Высокая стоимость подписки.
2. Непрозрачное поведение планировщика, который может менять план без явного контроля.
3. Достаточно высокий порог входа по UX и настройке.

6.3.1.2. Reclaim.ai

Плюсы:

1. Автоматический тайм-блокинг задач, привычек и фокуса.
2. Глубокая интеграция с Google Calendar, Slack и популярными таск-системами.
3. Календарный sync между несколькими календарями и защита личных слотов.
4. Аналитика и командные сценарии.

Минусы:

1. Жёсткая привязка к Google/Outlook, без прямой поддержки Apple Calendar.

2. Нет нативных приложений, фокус на веб/PWA.
3. Требует аккуратной первичной настройки рабочих часов и правил.

6.3.1.3. Sunsama

Плюсы:

1. Ежедневные ритуалы планирования и рефлексии.
2. Спокойная визуализация дня, сочетание задач и календаря.
3. Интеграции с популярными менеджерами задач.
4. Кроссплатформенность (веб + десктоп + мобильные приложения).

Минусы:

1. Отсутствие AI-автопланирования, упор на ручной контроль.
2. Высокая цена и отсутствие постоянного бесплатного тарифа.
3. Заметный объём ручных действий при плотном графике.

6.3.1.4. Things 3

Плюсы:

1. Нативный, быстрый и стабильный клиент в экосистеме Apple.
2. Удобная модель организации задач (Projects, Areas, Today, теги).
3. Надёжная офлайн-работа и простая синхронизация между устройствами Apple.

Минусы:

1. Почти нет прямых интеграций с внешними сервисами.
2. Нет продвинутой автоматизации и AI-логики, всё строится на ручных сценариях.

6.3.1.5. Todoist

Плюсы:

1. Мощные фильтры, метки и естественный ввод дат.
2. Кроссплатформенность и большая экосистема клиентов и расширений.
3. Интеграции с множеством внешних сервисов и автоматизаций.
4. Встроенная геймификация через систему Karma.

Минусы:

1. Продвинутые функции (фильтры, напоминания и др.) в платных тарифах.
2. При большом количестве проектов и меток интерфейс быстро захламляется.
3. Зависимость от облачной синхронизации.

6.3.1.6. Notion Calendar

Плюсы:

1. Глубокая связка с Notion и базами данных.
2. Совместная работа, шаринг и scheduling-линки.
3. Бесплатная часть экосистемы Notion.

Минусы:

1. Поддержка только Google Calendar, без нативного Apple/Outlook.
2. Относительно базовый функционал календаря по сравнению с профильными AI-решениями.
3. Просадки по производительности и удобству на крупных воркспейсах.

6.3.1.7. Sentinel

Плюсы:

1. Генерация нескольких событий из одного чат-запроса с предварительным просмотром.
2. Учёт «энергии дня» (Resource Battery) при подсказках и планировании.
3. Прозрачная ручная ребалансировка расписания.
4. Нативная интеграция с Apple Calendar через EventKit без сторонних облачных прослоек.
5. Экран достижений с геймификацией, завязанной на режим и энергию.

Минусы:

1. Отсутствие кроссплатформенности вне экосистемы Apple.
2. Нет глубоких интеграций с корпоративными таск-системами уровня Jira/Asana.

6.3.2. Дифференциаторы «Sentinel»

1. Генерация нескольких событий из одного чат-запроса с отметкой для добавления.
2. Учёт «энергии дня» (Resource Battery) при подсказках и планировании.
3. Автоматическая балансировка «Rebalance» для упорядочивания.
4. Нативная интеграция с Apple Calendar (EventKit) без сторонних облачных прослоек.
5. Система достижений с отдельным экраном «Достижения».

Расчёт экономической эффективности не производится в рамках курсового проекта.

6.3.3. Сравнительные критерии (матрица)

Критерий	Todoist	Google Calendar	Motion	Reclaim.ai	Sunsama	Things 3	Notion Calendar	Sentinel
Отображение событий	+	+	+	+	+	+	+	+
Генерация через ИИ	-	-	-	+	-	-	-	+
Интеграция с Apple Calendar	-	+	-	-	-	+	-	+
Ребалансировка расписания	-	-	+	+	-	-	-	+
Модель энергии дня	-	-	-	-	-	-	-	+

Критерий	Todoist	Google Calendar	Motion	Reclaim.ai	Sunsama	Things 3	Notion Calendar	Sentinel
Достижения/ геймификация	-	-	-	-	+	-	-	+
Кроссплатф.	+	+	+	+	+	-	+	-
Уведомления	+	+	+	+	+	+	+	+

Таблица 1. Сравнение функциональных характеристик

7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

7.1. Стадии разработки, этапы и содержание работ

Стадии и этапы разработки были выявлены с учётом ГОСТ 19.102-77 [2].

Стадия разработки	Этап работ	Содержание работ	Исполнители работ	Сроки выполнения
Техническое задание	Обоснование необходимости разработки	Постановка задачи	Петянов Т.А. и Максимов Т.С.	
		Сбор исходных теоретических материалов	Петянов Т.А. и Максимов Т.С.	
	Научно-исследовательский этап разработки	Определение структуры входных и выходных данных	Петянов Т.А. и Максимов Т.С.	
		Предварительный выбор методов решения задач	Петянов Т.А. и Максимов Т.С.	
		Определение требований к техническим и программным средствам	Петянов Т.А. и Максимов Т.С.	
		Обоснование возможности решения поставленной задачи	Петянов Т.А. и Максимов Т.С.	
	Разработка и утверждение технического задания	Определение требований к программному продукту	Петянов Т.А. и Максимов Т.С.	
		Выбор языков программирования	Петянов Т.А. и Максимов Т.С.	
		Разработка и согласование технического задания с научным руководителем	Петянов Т.А. и Максимов Т.С.	

Стадия разработки	Этап работ	Содержание работ	Исполнители работ	Сроки выполнения
		Загрузка согласованного технического задания в SmartLMS	Петянов Т.А. и Максимов Т.С.	

Таблица 2. Стадии и этапы разработки

Стадия разработки	Этап работ	Содержание работ	Исполнители работ	Сроки выполнения
Рабочий проект	Разработка программы	Предварительная разработка структуры программы	Петянов Т.А. и Максимов Т.С.	
		Программирование и отладка программы	Петянов Т.А. и Максимов Т.С.	
	Разработка программной документации	Разработка документов в соответствии с требованиями ГОСТ 19 ЕСПД (Единой системы программной документации)	Петянов Т.А. и Максимов Т.С.	
	Испытания программы	Разработка, согласование и утверждение порядка в методики испытаний	Петянов Т.А. и Максимов Т.С.	
		Проведение испытаний программы в соответствии с утвержденными порядком и методикой	Петянов Т.А. и Максимов Т.С.	
		Корректировка программы и программной	Петянов Т.А. и Максимов Т.С.	

Стадия разработки	Этап работ	Содержание работ	Исполнители работ	Сроки выполнения
		документации по результатам испытаний		
Разработка программы	Подготовка и передача программы	Подготовка программы и программной документации для презентации и защиты	Петянов Т.А. и Максимов Т.С.	
		Представление разработанного программного продукта научному руководителю и получение отзыва	Петянов Т.А. и Максимов Т.С.	

Стадия разработки	Этап работ	Содержание работ	Исполнители работ	Сроки выполнения
		Загрузка Пояснительной записки в систему Антиплагиат через ЛМС НИУ ВШЭ	Петянов Т.А. и Максимов Т.С.	
		Загрузка материалов курсового проекта в ЛМС, дисциплина «Курсовой проект, 3 курс, ПИ»	Петянов Т.А. и Максимов Т.С.	
		Защита программного продукта комиссии	Петянов Т.А. и Максимов Т.С.	

7.2. Сроки разработки и исполнители

Разработка должна быть завершена не позднее утвержденного срока сдачи курсового проекта. Исполнители: студенты группы БПИ236 Петянов Т.А. (Frontend/iOS) и Максимов Т.С. (Backend/ Python).

8. ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ

Контроль и приемка разработки осуществляются в соответствии с документом «Программа и методика испытаний» (ГОСТ 19.301–79 [8]).

8.1. Виды испытаний

Для проверки соответствия программы требованиям проводятся следующие виды испытаний:

1. Функциональные — чат, создание событий из ответов, работа с календарём.
2. Надёжность — устойчивость к сетевым сбоям и некорректным данным.
3. Производительность — время отклика интерфейса и обработки данных.
4. Удобство использования и доступность.

8.2. Общие требования к приемке работы

Работа считается выполненной при соблюдении следующих условий:

1. Успешная обработка не менее 90% текстовых запросов, фотографий и документов из тестового набора.
2. Отсутствие дублирования событий при экспорте в календарь.
3. Соблюдение временных ограничений на выполнение операций.
4. Отсутствие критических ошибок (Crash-free rate ≥ 99,5%).
5. Предоставление полного комплекта программной документации.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. Apple Human Interface Guidelines. URL: https://developer.apple.com/design/human-interface-guidelines/ (дата обращения: 29.11.2025).

ПРИЛОЖЕНИЕ А. ТЕРМИНОЛОГИЯ

1. **Event — Событие** — запись календаря с полями: заголовок, начало, окончание, заметки, идентификатор календаря.
2. **Resource Battery** — модель оценки текущего уровня энергии пользователя, учитывающая тип нагрузки и количество переключений контекста.
3. **Rebalance** — алгоритм автоматической оптимизации расписания путём перестановки событий.
4. **Chat — Чат** — диалог с ассистентом для получения предложений по событиям и действиям.
5. **EventKit** — системный фреймворк iOS для работы с календарями и напоминаниями.
6. **Apple Calendar** — системное приложение календаря в экосистеме Apple, используемое для хранения и отображения событий.
7. **Notion Calendar** — календарный продукт компании Notion, упомянутый в анализе аналогов.
8. **LLM (Large Language Model)** — большая языковая модель, используемая для интерпретации пользовательских запросов и извлечения сущностей.
9. **OpenAI SDK** — клиентская библиотека для обращения к внешним LLM-провайдерам из серверной части.
10. **JWT (JSON Web Token)** — формат токенов доступа для аутентификации и авторизации в API.
11. **ISO-8601** — международный стандарт представления даты и времени, применяемый в запросах и ответах API.
12. **IANA Time Zone Database** — база данных часовых поясов, используемая для корректного учёта пользовательской тайм-зоны.

ПРИЛОЖЕНИЕ Б. ССЫЛКИ НА АНАЛОГИ

Название	Ссылка
Motion	https://www.usemotion.com/
Reclaim.ai	https://reclaim.ai/
Sunsama	https://sunsama.com/
Things 3	https://culturedcode.com/things/
Todoist	https://todoist.com/
Notion Calendar	https://www.notion.so/product/calendar
Google Calendar	https://workspace.google.com/products/calendar/

Дата обращения: 29.11.2025.

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц в докум.)	№ документа	Входящий № сопроводительного докум. и дата	Подп.	Дата
Изм.	Измененных	Замененных	Новых	Аннулированных					